Replit Development Prompt 12a: Staged Document Requirements System

Prerequisites

- Prompts 11, 11a, and 11b completed with document upload, portal, and management systems
- Pipeline management system operational (Prompts 6-10)
- Target company and contact management working
- Document requirements tracking foundation established

What This Prompt Builds

Implement a sophisticated two-stage document collection system that aligns with the M&A process workflow. Stage 1 (Pre-Offer) focuses on initial due diligence documents needed for valuation and offer preparation. Stage 2 (Post-Heads of Terms) covers comprehensive due diligence documents required for final legal and accounting processes. This system ensures proper document flow and prevents premature progression through the M&A pipeline.

System Context Reminder

- **Stage 1 Documents**: Initial due diligence required before offer preparation (financial statements, key contracts, management information)
- **Stage 2 Documents**: Comprehensive due diligence required after heads of terms acceptance (detailed legal, tax, HR, insurance documentation)
- **Pipeline Integration**: Document stage completion controls pipeline progression
- **Industry Customization**: Different document requirements based on target company industry

What You Need to Build

1. Staged Document Requirements System

Create comprehensive staged document collection with these capabilities:

Two-Stage Document Collection:

- Stage 1: Pre-Offer due diligence (essential documents for valuation)

- Stage 2: Post-Heads of Terms due diligence (comprehensive legal and operational documents)
- Stage progression controls preventing premature pipeline advancement
- Industry-specific document requirement templates

Document Requirement Management:

- Customizable document checklists by industry and deal size
- Mandatory vs. optional document classification
- Document interdependencies and prerequisites
- Waiver system with justification requirements

Pipeline Integration:

- Stage 1 completion required to reach "Offer Preparation" pipeline stage
- Stage 2 completion required to reach "Legal & Accounting" pipeline stage
- Automatic stage progression triggers based on document validation
- Override capabilities for exceptional circumstances

2. Database Schema for Staged Requirements

```sql
-- Create document requirement templates table
CREATE TABLE document_requirement_templates (
  id SERIAL PRIMARY KEY,
  user_company_id INTEGER REFERENCES user_companies(id),
  created_by INTEGER REFERENCES users(id),

  -- Template Details
  template_name VARCHAR(255) NOT NULL,
  template_description TEXT,
  industry_category VARCHAR(100), -- technology, manufacturing, retail,
services, healthcare, etc.
  deal_size_category VARCHAR(50), -- small, medium, large, enterprise

  -- Template Configuration
  is_default BOOLEAN DEFAULT FALSE,
  is_active BOOLEAN DEFAULT TRUE,
  template_version VARCHAR(20) DEFAULT '1.0',

  -- Usage Tracking
  usage_count INTEGER DEFAULT 0,
  last_used TIMESTAMP,

  created_at TIMESTAMP DEFAULT NOW(),
```

```sql
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Create document requirement items table
CREATE TABLE document_requirement_items (
  id SERIAL PRIMARY KEY,
  template_id INTEGER REFERENCES document_requirement_templates(id),

  -- Requirement Details
  requirement_name VARCHAR(255) NOT NULL,
  requirement_description TEXT,
  document_category VARCHAR(100), -- financial, legal, operational, commercial,
hr, tax, insurance

  -- Stage and Priority
  collection_stage INTEGER NOT NULL, -- 1 = pre-offer, 2 = post-heads-of-terms
  priority_level VARCHAR(20) DEFAULT 'medium', -- low, medium, high, critical
  is_mandatory BOOLEAN DEFAULT TRUE,

  -- Document Specifications
  accepted_file_types TEXT[] DEFAULT ARRAY['pdf', 'docx', 'xlsx'], -- Accepted
file formats
  max_file_size BIGINT DEFAULT 10485760, -- 10MB default
  min_file_size BIGINT DEFAULT 1024, -- 1KB minimum
  max_file_count INTEGER DEFAULT 1, -- Maximum number of files for this
requirement

  -- Instructions and Help
  submission_instructions TEXT,
  example_documents TEXT[], -- URLs or descriptions of example documents
  help_text TEXT,
  common_issues TEXT, -- Common problems and solutions

  -- Dependencies
  prerequisite_requirements INTEGER[], -- Array of requirement IDs that must be
completed first
  related_requirements INTEGER[], -- Array of related requirement IDs

  -- Validation Rules
  validation_rules JSONB, -- Custom validation rules for this document type
  auto_validation BOOLEAN DEFAULT FALSE, -- Whether this can be auto-validated

  -- Display and Ordering
  display_order INTEGER DEFAULT 0,
  is_visible BOOLEAN DEFAULT TRUE,

  created_at TIMESTAMP DEFAULT NOW(),
```

```
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Create company document requirements table (instance of template for
specific deal)
CREATE TABLE company_document_requirements (
  id SERIAL PRIMARY KEY,
  target_company_id INTEGER REFERENCES target_companies(id),
  template_id INTEGER REFERENCES document_requirement_templates(id),
  requirement_item_id INTEGER REFERENCES document_requirement_items(id),

  -- Requirement Status
  requirement_status VARCHAR(50) DEFAULT 'outstanding', -- outstanding,
submitted, under_review, approved, rejected, waived
  submission_status VARCHAR(50) DEFAULT 'not_started', -- not_started,
in_progress, completed, overdue

  -- Submission Details
  submitted_document_id INTEGER REFERENCES external_document_submissions(id),
  submitted_at TIMESTAMP,
  submitted_by_contact_id INTEGER REFERENCES target_contacts(id),

  -- Review and Approval
  reviewed_by INTEGER REFERENCES users(id),
  reviewed_at TIMESTAMP,
  review_status VARCHAR(50), -- pending, approved, rejected, needs_revision
  review_notes TEXT,
  review_score INTEGER, -- 1-5 quality score

  -- Waiver Information
  is_waived BOOLEAN DEFAULT FALSE,
  waived_by INTEGER REFERENCES users(id),
  waived_at TIMESTAMP,
  waiver_reason TEXT,
  waiver_justification TEXT,
  waiver_approved_by INTEGER REFERENCES users(id),

  -- Due Dates and Reminders
  due_date TIMESTAMP,
  reminder_sent_count INTEGER DEFAULT 0,
  last_reminder_sent TIMESTAMP,
  next_reminder_due TIMESTAMP,

  -- Custom Modifications
  custom_instructions TEXT, -- Deal-specific instructions
  custom_priority VARCHAR(20), -- Override template priority
```

```
  is_custom_requirement BOOLEAN DEFAULT FALSE, -- Added specifically for this
deal

  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Create document stage progression table
CREATE TABLE document_stage_progression (
  id SERIAL PRIMARY KEY,
  target_company_id INTEGER REFERENCES target_companies(id),

  -- Stage Information
  current_stage INTEGER DEFAULT 1, -- 1 = stage 1, 2 = stage 2
  stage_1_status VARCHAR(50) DEFAULT 'in_progress', -- not_started,
in_progress, completed, blocked
  stage_2_status VARCHAR(50) DEFAULT 'not_started', -- not_started,
in_progress, completed, blocked

  -- Stage 1 Completion
  stage_1_completed_at TIMESTAMP,
  stage_1_completed_by INTEGER REFERENCES users(id),
  stage_1_completion_percentage DECIMAL(5,2) DEFAULT 0.00,
  stage_1_mandatory_completed BOOLEAN DEFAULT FALSE,

  -- Stage 2 Completion
  stage_2_started_at TIMESTAMP,
  stage_2_completed_at TIMESTAMP,
  stage_2_completed_by INTEGER REFERENCES users(id),
  stage_2_completion_percentage DECIMAL(5,2) DEFAULT 0.00,
  stage_2_mandatory_completed BOOLEAN DEFAULT FALSE,

  -- Overall Progress
  total_requirements INTEGER DEFAULT 0,
  completed_requirements INTEGER DEFAULT 0,
  overall_completion_percentage DECIMAL(5,2) DEFAULT 0.00,

  -- Pipeline Integration
  can_progress_to_offer BOOLEAN DEFAULT FALSE,
  can_progress_to_legal BOOLEAN DEFAULT FALSE,
  pipeline_stage_updated_at TIMESTAMP,

  -- Blocking Issues
  blocking_issues JSONB, -- Array of issues preventing progression
  resolution_notes TEXT,

  created_at TIMESTAMP DEFAULT NOW(),
```

```
  updated_at TIMESTAMP DEFAULT NOW(),

  UNIQUE(target_company_id)
);

-- Create document requirement waivers table
CREATE TABLE document_requirement_waivers (
  id SERIAL PRIMARY KEY,
  company_requirement_id INTEGER REFERENCES company_document_requirements(id),
  target_company_id INTEGER REFERENCES target_companies(id),

  -- Waiver Request
  requested_by INTEGER REFERENCES users(id),
  requested_at TIMESTAMP DEFAULT NOW(),
  waiver_reason VARCHAR(100), -- not_applicable, not_available, confidential,
alternative_provided
  waiver_justification TEXT NOT NULL,
  alternative_documentation TEXT, -- Description of alternative documents
provided

  -- Approval Process
  approval_status VARCHAR(50) DEFAULT 'pending', -- pending, approved,
rejected, escalated
  approved_by INTEGER REFERENCES users(id),
  approved_at TIMESTAMP,
  approval_notes TEXT,

  -- Risk Assessment
  risk_level VARCHAR(20) DEFAULT 'low', -- low, medium, high, critical
  risk_assessment TEXT,
  mitigation_measures TEXT,

  -- Conditions
  conditional_approval BOOLEAN DEFAULT FALSE,
  approval_conditions TEXT,
  condition_met BOOLEAN DEFAULT FALSE,
  condition_verified_by INTEGER REFERENCES users(id),
  condition_verified_at TIMESTAMP,

  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Insert default document requirement templates
INSERT INTO document_requirement_templates (user_company_id, template_name,
template_description, industry_category, deal_size_category, is_default) VALUES
```

```
(1, 'Technology Company - Standard', 'Standard document requirements for
technology company acquisitions', 'technology', 'medium', TRUE),
(1, 'Manufacturing Company - Standard', 'Standard document requirements for
manufacturing company acquisitions', 'manufacturing', 'medium', TRUE),
(1, 'Service Company - Standard', 'Standard document requirements for service
company acquisitions', 'services', 'medium', TRUE),
(1, 'Retail Company - Standard', 'Standard document requirements for retail
company acquisitions', 'retail', 'medium', TRUE),
(1, 'Healthcare Company - Standard', 'Standard document requirements for
healthcare company acquisitions', 'healthcare', 'medium', TRUE);

-- Insert Stage 1 (Pre-Offer) document requirements for Technology template
INSERT INTO document_requirement_items (template_id, requirement_name,
requirement_description, document_category, collection_stage, priority_level,
is_mandatory, submission_instructions, help_text, display_order) VALUES
-- Financial Documents (Stage 1)
(1, 'Audited Financial Statements (3 Years)', 'Audited financial statements for
the last three complete financial years including Profit & Loss, Balance Sheet,
and Cash Flow statements', 'financial', 1, 'critical', TRUE, 'Please provide
audited financial statements prepared by a qualified accountant. If audited
statements are not available, management accounts prepared by a qualified
accountant are acceptable.', 'These documents are essential for financial due
diligence and company valuation. They should include all subsidiaries and show
a true and fair view of the company''s financial position.', 1),

(1, 'Management Accounts (Current Year)', 'Latest management accounts showing
current year performance to date', 'financial', 1, 'critical', TRUE, 'Please
provide the most recent management accounts available, ideally not more than 3
months old. Include monthly or quarterly breakdowns if available.', 'Management
accounts help us understand current trading performance and identify any recent
trends that may not be reflected in the audited accounts.', 2),

(1, 'Financial Projections/Budget', 'Financial projections or budget for the
current and next financial year', 'financial', 1, 'high', TRUE, 'Please provide
detailed financial projections including assumptions. If formal projections are
not available, please provide budget information or management estimates.',
'Projections help us understand management''s expectations for future
performance and validate our valuation assumptions.', 3),

(1, 'Bank Statements (6 Months)', 'Bank statements for all business accounts
for the last 6 months', 'financial', 1, 'high', TRUE, 'Please provide
statements for all business bank accounts. Personal account statements are not
required unless business transactions are conducted through personal
accounts.', 'Bank statements help verify cash flow and identify any unusual
transactions or financing arrangements.', 4),

-- Commercial Documents (Stage 1)
```

```
(1, 'Customer Analysis', 'Analysis of top customers including revenue
contribution and contract terms', 'commercial', 1, 'high', TRUE, 'Please
provide a list of your top 10-20 customers showing annual revenue, contract
terms, and payment history. Include any key customer contracts or agreements.',
'Customer concentration analysis is critical for assessing business risk and
revenue sustainability.', 5),

(1, 'Supplier Analysis', 'Analysis of key suppliers and supply chain
arrangements', 'commercial', 1, 'medium', FALSE, 'Please provide details of
your main suppliers, including spend levels, contract terms, and any critical
dependencies.', 'Understanding supplier relationships helps assess operational
risk and potential supply chain issues.', 6),

(1, 'Sales and Marketing Information', 'Information about sales processes,
marketing activities, and market position', 'commercial', 1, 'medium', TRUE,
'Please provide information about your sales and marketing activities,
including sales processes, marketing spend, and competitive position.', 'This
information helps us understand the commercial drivers of the business and
growth potential.', 7),

-- Operational Documents (Stage 1)
(1, 'Organizational Chart', 'Current organizational structure showing key
personnel and reporting relationships', 'operational', 1, 'medium', TRUE,
'Please provide an organizational chart showing all key personnel, their roles,
and reporting relationships. Include brief CVs for senior management.',
'Understanding the management structure and key personnel is essential for
assessing operational capability and succession planning.', 8),

(1, 'Key Management Information', 'Information about key management personnel
including CVs and service agreements', 'operational', 1, 'medium', TRUE,
'Please provide CVs for all senior management and details of their service
agreements, including notice periods and any restrictive covenants.', 'Key
person risk is a critical factor in business valuation and post-acquisition
planning.', 9),

(1, 'Business Overview', 'Comprehensive overview of the business including
products, services, and operations', 'operational', 1, 'high', TRUE, 'Please
provide a detailed description of your business including products/services
offered, operational processes, and key business drivers.', 'A clear
understanding of the business model and operations is fundamental to our
evaluation process.', 10);

-- Insert Stage 2 (Post-Heads of Terms) document requirements for Technology
template
INSERT INTO document_requirement_items (template_id, requirement_name,
requirement_description, document_category, collection_stage, priority_level,
is_mandatory, submission_instructions, help_text, display_order) VALUES
```

```
-- Legal Documents (Stage 2)
(1, 'Constitutional Documents', 'Articles of incorporation, memorandum of
association, and all amendments', 'legal', 2, 'critical', TRUE, 'Please provide
all constitutional documents including any amendments or resolutions that have
modified the original documents.', 'These documents establish the legal
structure of the company and any restrictions on its activities or share
transfers.', 1),

(1, 'Shareholder Agreements', 'All shareholder agreements, voting agreements,
and share option schemes', 'legal', 2, 'critical', TRUE, 'Please provide all
agreements between shareholders including any drag-along, tag-along, or
pre-emption rights.', 'Shareholder agreements may contain provisions that
affect the acquisition process or post-acquisition governance.', 2),

(1, 'Material Contracts', 'All material contracts including customer
agreements, supplier contracts, and partnership agreements', 'legal', 2,
'critical', TRUE, 'Please provide all contracts with annual value exceeding
£50,000 or that are otherwise material to the business. Include any contracts
with change of control provisions.', 'Material contracts may contain change of
control clauses or other provisions that could be triggered by the
acquisition.', 3),

(1, 'Intellectual Property Documentation', 'Patents, trademarks, copyrights,
and licensing agreements', 'legal', 2, 'high', TRUE, 'Please provide details of
all intellectual property owned or licensed by the company, including
registration certificates and licensing agreements.', 'Intellectual property is
often a key asset in technology companies and must be properly documented and
protected.', 4),

(1, 'Litigation and Disputes', 'Details of any current or threatened
litigation, disputes, or regulatory investigations', 'legal', 2, 'high', TRUE,
'Please provide details of any legal proceedings, disputes, or regulatory
investigations, including correspondence with lawyers and any settlement
agreements.', 'Legal issues can represent significant liabilities and must be
fully disclosed and assessed.', 5),

-- HR Documents (Stage 2)
(1, 'Employee Contracts', 'Employment contracts for all employees, including
directors and senior management', 'hr', 2, 'high', TRUE, 'Please provide
employment contracts for all staff, including any variations or amendments.
Include details of notice periods, restrictive covenants, and termination
provisions.', 'Employment contracts may contain provisions that affect
post-acquisition integration and employee retention.', 6),

(1, 'Employee Handbook and Policies', 'Employee handbook, HR policies, and
procedures', 'hr', 2, 'medium', TRUE, 'Please provide the employee handbook and
all HR policies including disciplinary procedures, grievance procedures, and
```

```
health and safety policies.', 'HR policies help us understand employment
practices and identify any compliance issues.', 7),

(1, 'Pension and Benefits Information', 'Details of pension schemes, employee
benefits, and any employee share schemes', 'hr', 2, 'high', TRUE, 'Please
provide details of all employee benefits including pension schemes, health
insurance, and any employee share or option schemes.', 'Employee benefits
represent ongoing liabilities that must be factored into the acquisition cost
and post-acquisition planning.', 8),

-- Tax Documents (Stage 2)
(1, 'Tax Returns and Computations', 'Corporation tax returns and computations
for the last 3 years', 'tax', 2, 'high', TRUE, 'Please provide corporation tax
returns and supporting computations for the last three years, including any
amendments or enquiries.', 'Tax returns help verify the tax position and
identify any potential tax liabilities or opportunities.', 9),

(1, 'VAT Returns and Records', 'VAT returns and supporting records for the last
2 years', 'tax', 2, 'medium', TRUE, 'Please provide VAT returns and supporting
records, including any correspondence with HMRC regarding VAT matters.', 'VAT
compliance is important for ongoing operations and any errors could result in
penalties or additional liabilities.', 10),

(1, 'Tax Correspondence', 'All correspondence with tax authorities including
enquiries, investigations, and settlements', 'tax', 2, 'high', TRUE, 'Please
provide all correspondence with HMRC or other tax authorities, including any
ongoing enquiries or investigations.', 'Tax enquiries or investigations could
result in additional liabilities and must be fully understood.', 11),

-- Insurance Documents (Stage 2)
(1, 'Insurance Policies', 'All current insurance policies including
professional indemnity, public liability, and directors and officers
insurance', 'insurance', 2, 'medium', TRUE, 'Please provide copies of all
current insurance policies and details of any claims made in the last 3
years.', 'Insurance coverage helps protect against various business risks and
claims history may indicate potential issues.', 12),

(1, 'Insurance Claims History', 'Details of all insurance claims made in the
last 5 years', 'insurance', 2, 'medium', FALSE, 'Please provide details of all
insurance claims including amounts claimed, amounts received, and current
status of any outstanding claims.', 'Claims history helps assess risk profile
and may indicate operational or other issues.', 13);

-- Create indexes for performance
CREATE INDEX idx_requirement_templates_industry ON
document_requirement_templates(industry_category, is_active);
```

```
CREATE INDEX idx_requirement_items_template ON
document_requirement_items(template_id, collection_stage);
CREATE INDEX idx_company_requirements_company ON
company_document_requirements(target_company_id, requirement_status);
CREATE INDEX idx_company_requirements_stage ON
company_document_requirements(target_company_id, requirement_item_id);
CREATE INDEX idx_stage_progression_company ON
document_stage_progression(target_company_id);
CREATE INDEX idx_requirement_waivers_company ON
document_requirement_waivers(target_company_id, approval_status);
```

3. API Endpoints for Staged Requirements

JavaScript

```
// Document requirement templates
GET /api/document-requirements/templates          // Get requirement templates
POST /api/document-requirements/templates         // Create custom template
PUT /api/document-requirements/templates/:id      // Update template
DELETE /api/document-requirements/templates/:id   // Delete template

// Company document requirements
GET /api/companies/:id/requirements               // Get requirements for
company
POST /api/companies/:id/requirements/initialize   // Initialize requirements
from template
PUT /api/companies/:id/requirements/:reqId        // Update specific
requirement
POST /api/companies/:id/requirements/waiver       // Request requirement waiver

// Stage progression
GET /api/companies/:id/stage-progression          // Get stage progression
status
POST /api/companies/:id/stage-progression/advance // Advance to next stage
PUT /api/companies/:id/stage-progression/override // Override stage
requirements

// Document validation and review
POST /api/document-requirements/review            // Submit document for review
PUT /api/document-requirements/review/:id         // Update review status
GET /api/document-requirements/pending-review     // Get documents pending
review

// Requirement waivers
GET /api/document-requirements/waivers            // Get waiver requests
POST /api/document-requirements/waivers           // Create waiver request
```

```
PUT /api/document-requirements/waivers/:id        // Approve/reject waiver
```

4. Staged Requirements Implementation

JavaScript

```
const stagedRequirementsManager = {
  // Initialize document requirements for a company
  initializeCompanyRequirements: async (targetCompanyId, templateId, createdBy)
=> {
    // Verify permissions
    const user = await getUserWithPermissions(createdBy);
    if (!user.can_create_deals && !user.can_manage_company) {
      throw new Error('Insufficient permissions to initialize document
requirements');
    }

    // Get company information for industry matching
    const company = await db.query('SELECT * FROM target_companies WHERE id =
$1', [targetCompanyId]);
    if (company.rows.length === 0) {
      throw new Error('Target company not found');
    }

    const companyData = company.rows[0];

    // If no template specified, find default template for industry
    let selectedTemplateId = templateId;
    if (!selectedTemplateId) {
      const defaultTemplate = await db.query(`
        SELECT id FROM document_requirement_templates
        WHERE industry_category = $1 AND is_default = true AND is_active = true
        LIMIT 1
      `, [companyData.industry?.toLowerCase() || 'general']);

      if (defaultTemplate.rows.length > 0) {
        selectedTemplateId = defaultTemplate.rows[0].id;
      } else {
        // Fall back to general template
        const generalTemplate = await db.query(`
          SELECT id FROM document_requirement_templates
          WHERE is_default = true AND is_active = true
          LIMIT 1
        `);
        selectedTemplateId = generalTemplate.rows[0]?.id;
      }
```

```
    }

    if (!selectedTemplateId) {
      throw new Error('No suitable document requirement template found');
    }

    // Get template requirements
    const templateRequirements = await db.query(`
      SELECT * FROM document_requirement_items
      WHERE template_id = $1 AND is_visible = true
      ORDER BY collection_stage ASC, display_order ASC
    `, [selectedTemplateId]);

    // Create company requirements from template
    const createdRequirements = [];
    let totalRequirements = 0;

    for (const templateReq of templateRequirements.rows) {
      const requirementQuery = `
        INSERT INTO company_document_requirements (
          target_company_id, template_id, requirement_item_id,
          due_date, next_reminder_due
        ) VALUES ($1, $2, $3, $4, $5)
        RETURNING *
      `;

      // Set due dates based on stage and priority
      const dueDate = calculateDueDate(templateReq.collection_stage,
templateReq.priority_level);
      const reminderDate = calculateReminderDate(dueDate);

      const values = [
        targetCompanyId,
        selectedTemplateId,
        templateReq.id,
        dueDate,
        reminderDate
      ];

      const result = await db.query(requirementQuery, values);
      createdRequirements.push(result.rows[0]);
      totalRequirements++;
    }

    // Initialize stage progression
    await db.query(`
      INSERT INTO document_stage_progression (
```

```
        target_company_id, total_requirements
      ) VALUES ($1, $2)
      ON CONFLICT (target_company_id)
      DO UPDATE SET total_requirements = EXCLUDED.total_requirements
    `, [targetCompanyId, totalRequirements]);

    // Log activity
    await logActivity(targetCompanyId, createdBy,
'document_requirements_initialized', `Initialized ${totalRequirements} document
requirements`);

    return {
      template_id: selectedTemplateId,
      requirements_created: createdRequirements.length,
      total_requirements: totalRequirements
    };
  },

  // Get document requirements for a company by stage
  getCompanyRequirements: async (targetCompanyId, stage = null, status = null)
=> {
    let requirementsQuery = `
      SELECT
        cdr.*,
        dri.requirement_name,
        dri.requirement_description,
        dri.document_category,
        dri.collection_stage,
        dri.priority_level,
        dri.is_mandatory,
        dri.accepted_file_types,
        dri.max_file_size,
        dri.submission_instructions,
        dri.help_text,
        dri.display_order,
        eds.original_filename,
        eds.submitted_at as document_submitted_at,
        eds.processing_status as document_processing_status,
        tc.first_name as contact_first_name,
        tc.last_name as contact_last_name
      FROM company_document_requirements cdr
      JOIN document_requirement_items dri ON cdr.requirement_item_id = dri.id
      LEFT JOIN external_document_submissions eds ON cdr.submitted_document_id
= eds.id
      LEFT JOIN target_contacts tc ON cdr.submitted_by_contact_id = tc.id
      WHERE cdr.target_company_id = $1
    `;
```

```
    const queryParams = [targetCompanyId];

    if (stage !== null) {
      requirementsQuery += ' AND dri.collection_stage = $' +
(queryParams.length + 1);
      queryParams.push(stage);
    }

    if (status !== null) {
      requirementsQuery += ' AND cdr.requirement_status = $' +
(queryParams.length + 1);
      queryParams.push(status);
    }

    requirementsQuery += ' ORDER BY dri.collection_stage ASC, dri.display_order
ASC, dri.priority_level DESC';

    const result = await db.query(requirementsQuery, queryParams);
    return result.rows;
  },

  // Update stage progression when documents are submitted/approved
  updateStageProgression: async (targetCompanyId) => {
    // Calculate completion statistics
    const progressQuery = `
      SELECT
        dri.collection_stage,
        COUNT(*) as total_requirements,
        COUNT(CASE WHEN cdr.requirement_status = 'approved' THEN 1 END) as
approved_count,
        COUNT(CASE WHEN cdr.requirement_status IN ('submitted', 'under_review',
'approved') THEN 1 END) as submitted_count,
        COUNT(CASE WHEN dri.is_mandatory = true THEN 1 END) as mandatory_total,
        COUNT(CASE WHEN dri.is_mandatory = true AND cdr.requirement_status =
'approved' THEN 1 END) as mandatory_approved,
        COUNT(CASE WHEN cdr.is_waived = true THEN 1 END) as waived_count
      FROM company_document_requirements cdr
      JOIN document_requirement_items dri ON cdr.requirement_item_id = dri.id
      WHERE cdr.target_company_id = $1
      GROUP BY dri.collection_stage
    `;

    const progressResult = await db.query(progressQuery, [targetCompanyId]);
    const stageStats = {};

    progressResult.rows.forEach(row => {
```

```
      const stage = row.collection_stage;
      const completionPercentage = row.total_requirements > 0
        ? Math.round(((parseInt(row.approved_count) +
parseInt(row.waived_count)) / parseInt(row.total_requirements)) * 100)
        : 0;

      const mandatoryCompleted = (parseInt(row.mandatory_approved) +
parseInt(row.waived_count)) >= parseInt(row.mandatory_total);

      stageStats[stage] = {
        total: parseInt(row.total_requirements),
        approved: parseInt(row.approved_count),
        submitted: parseInt(row.submitted_count),
        waived: parseInt(row.waived_count),
        mandatory_total: parseInt(row.mandatory_total),
        mandatory_approved: parseInt(row.mandatory_approved),
        completion_percentage: completionPercentage,
        mandatory_completed: mandatoryCompleted
      };
    });

    // Update stage progression record
    const stage1Stats = stageStats[1] || { completion_percentage: 0,
mandatory_completed: false };
    const stage2Stats = stageStats[2] || { completion_percentage: 0,
mandatory_completed: false };

    const totalRequirements = (stage1Stats.total || 0) + (stage2Stats.total ||
0);
    const totalApproved = (stage1Stats.approved || 0) + (stage2Stats.approved
|| 0) + (stage1Stats.waived || 0) + (stage2Stats.waived || 0);
    const overallCompletion = totalRequirements > 0 ? Math.round((totalApproved
/ totalRequirements) * 100) : 0;

    // Determine current stage and progression capabilities
    let currentStage = 1;
    let canProgressToOffer = false;
    let canProgressToLegal = false;

    if (stage1Stats.mandatory_completed) {
      canProgressToOffer = true;
      if (stage2Stats.mandatory_completed) {
        currentStage = 2;
        canProgressToLegal = true;
      }
    }
```

```
    const updateQuery = `
      UPDATE document_stage_progression SET
        current_stage = $2,
        stage_1_completion_percentage = $3,
        stage_1_mandatory_completed = $4,
        stage_2_completion_percentage = $5,
        stage_2_mandatory_completed = $6,
        total_requirements = $7,
        completed_requirements = $8,
        overall_completion_percentage = $9,
        can_progress_to_offer = $10,
        can_progress_to_legal = $11,
        updated_at = NOW()
      WHERE target_company_id = $1
    `;

    await db.query(updateQuery, [
      targetCompanyId,
      currentStage,
      stage1Stats.completion_percentage,
      stage1Stats.mandatory_completed,
      stage2Stats.completion_percentage,
      stage2Stats.mandatory_completed,
      totalRequirements,
      totalApproved,
      overallCompletion,
      canProgressToOffer,
      canProgressToLegal
    ]);

    // Update pipeline stage if progression is possible
    await checkPipelineProgression(targetCompanyId, canProgressToOffer,
canProgressToLegal);

    return {
      current_stage: currentStage,
      stage_1_completion: stage1Stats.completion_percentage,
      stage_2_completion: stage2Stats.completion_percentage,
      overall_completion: overallCompletion,
      can_progress_to_offer: canProgressToOffer,
      can_progress_to_legal: canProgressToLegal
    };
  },

  // Process document submission and update requirements
  processDocumentSubmission: async (submissionId, requirementId) => {
    // Get submission details
```

```
    const submission = await db.query('SELECT * FROM
external_document_submissions WHERE id = $1', [submissionId]);
    if (submission.rows.length === 0) {
      throw new Error('Document submission not found');
    }

    const submissionData = submission.rows[0];

    // Update company requirement
    await db.query(`
      UPDATE company_document_requirements SET
        requirement_status = 'submitted',
        submission_status = 'completed',
        submitted_document_id = $2,
        submitted_at = NOW(),
        submitted_by_contact_id = $3
      WHERE id = $1
    `, [requirementId, submissionId, submissionData.target_contact_id]);

    // Update stage progression
    await
stagedRequirementsManager.updateStageProgression(submissionData.target_company_
id);

    // Create notification for internal team
    await notifyInternalTeam(
      submissionData.target_company_id,
      'document_requirement_submitted',
      `Document submitted for requirement:
${submissionData.original_filename}`,
      submissionId
    );

    return true;
  },

  // Request waiver for document requirement
  requestRequirementWaiver: async (companyRequirementId, waiverData,
requestedBy) => {
    // Verify permissions
    const user = await getUserWithPermissions(requestedBy);
    if (!user.can_manage_company && !user.can_edit_all_deals) {
      throw new Error('Insufficient permissions to request requirement
waiver');
    }

    // Get requirement details
```

```
    const requirement = await db.query(`
      SELECT cdr.*, dri.requirement_name, dri.is_mandatory,
dri.collection_stage
      FROM company_document_requirements cdr
      JOIN document_requirement_items dri ON cdr.requirement_item_id = dri.id
      WHERE cdr.id = $1
    `, [companyRequirementId]);

    if (requirement.rows.length === 0) {
      throw new Error('Document requirement not found');
    }

    const reqData = requirement.rows[0];

    // Create waiver request
    const waiverQuery = `
      INSERT INTO document_requirement_waivers (
        company_requirement_id, target_company_id, requested_by,
        waiver_reason, waiver_justification, alternative_documentation,
        risk_level, risk_assessment
      ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
      RETURNING *
    `;

    const waiverValues = [
      companyRequirementId,
      reqData.target_company_id,
      requestedBy,
      waiverData.waiver_reason,
      waiverData.waiver_justification,
      waiverData.alternative_documentation || '',
      waiverData.risk_level || 'medium',
      waiverData.risk_assessment || ''
    ];

    const waiverResult = await db.query(waiverQuery, waiverValues);
    const waiver = waiverResult.rows[0];

    // Update requirement status
    await db.query(
      'UPDATE company_document_requirements SET requirement_status = $1 WHERE
id = $2',
      ['waiver_requested', companyRequirementId]
    );

    // Log activity
```

```
    await logActivity(reqData.target_company_id, requestedBy,
'requirement_waiver_requested', `Waiver requested for:
${reqData.requirement_name}`);

    // Notify approvers
    await notifyWaiverApprovers(reqData.target_company_id, waiver.id,
reqData.requirement_name);

    return waiver;
  },

  // Approve or reject waiver request
  processWaiverRequest: async (waiverId, approvalData, approvedBy) => {
    // Verify permissions
    const user = await getUserWithPermissions(approvedBy);
    if (!user.can_manage_company && !user.is_company_admin) {
      throw new Error('Insufficient permissions to approve waivers');
    }

    // Get waiver details
    const waiver = await db.query(`
      SELECT drw.*, cdr.target_company_id, dri.requirement_name
      FROM document_requirement_waivers drw
      JOIN company_document_requirements cdr ON drw.company_requirement_id =
cdr.id
      JOIN document_requirement_items dri ON cdr.requirement_item_id = dri.id
      WHERE drw.id = $1
    `, [waiverId]);

    if (waiver.rows.length === 0) {
      throw new Error('Waiver request not found');
    }

    const waiverData = waiver.rows[0];

    // Update waiver status
    const updateWaiverQuery = `
      UPDATE document_requirement_waivers SET
        approval_status = $2,
        approved_by = $3,
        approved_at = NOW(),
        approval_notes = $4,
        conditional_approval = $5,
        approval_conditions = $6
      WHERE id = $1
      RETURNING *
    `;
```

```
    const waiverValues = [
      waiverId,
      approvalData.approval_status,
      approvedBy,
      approvalData.approval_notes || '',
      approvalData.conditional_approval || false,
      approvalData.approval_conditions || ''
    ];

    const updatedWaiver = await db.query(updateWaiverQuery, waiverValues);

    // Update company requirement based on approval
    let requirementStatus = 'outstanding';
    if (approvalData.approval_status === 'approved') {
      requirementStatus = 'waived';

      // Update company requirement
      await db.query(`
        UPDATE company_document_requirements SET
          requirement_status = 'waived',
          is_waived = true,
          waived_by = $2,
          waived_at = NOW(),
          waiver_reason = $3,
          waiver_justification = $4
        WHERE id = $1
      `, [waiverData.company_requirement_id, approvedBy,
waiverData.waiver_reason, waiverData.waiver_justification]);

      // Update stage progression
      await
stagedRequirementsManager.updateStageProgression(waiverData.target_company_id);

    } else if (approvalData.approval_status === 'rejected') {
      requirementStatus = 'outstanding';

      await db.query(
        'UPDATE company_document_requirements SET requirement_status = $1 WHERE
id = $2',
        [requirementStatus, waiverData.company_requirement_id]
      );
    }

    // Log activity
    const activityMessage = `Waiver ${approvalData.approval_status} for:
${waiverData.requirement_name}`;
```

```
    await logActivity(waiverData.target_company_id, approvedBy,
'requirement_waiver_processed', activityMessage);

    // Notify requester
    await createTeamNotification(
      waiverData.requested_by,
      'waiver_processed',
      `Waiver ${approvalData.approval_status}`,
      `Your waiver request for "${waiverData.requirement_name}" has been
${approvalData.approval_status}`,
      waiverData.target_company_id,
      approvedBy
    );

    return updatedWaiver.rows[0];
  }
};

const documentTemplateManager = {
  // Create custom document requirement template
  createRequirementTemplate: async (templateData, createdBy) => {
    const user = await getUserWithPermissions(createdBy);
    if (!user.can_manage_company && !user.is_company_admin) {
      throw new Error('Insufficient permissions to create requirement
templates');
    }

    // Create template
    const templateQuery = `
      INSERT INTO document_requirement_templates (
        user_company_id, created_by, template_name, template_description,
        industry_category, deal_size_category
      ) VALUES ($1, $2, $3, $4, $5, $6)
      RETURNING *
    `;

    const templateValues = [
      user.user_company_id,
      createdBy,
      templateData.template_name,
      templateData.template_description || '',
      templateData.industry_category || 'general',
      templateData.deal_size_category || 'medium'
    ];

    const templateResult = await db.query(templateQuery, templateValues);
    const template = templateResult.rows[0];
```

```
    // Add requirement items if provided
    if (templateData.requirements && templateData.requirements.length > 0) {
      for (const req of templateData.requirements) {
        await addRequirementToTemplate(template.id, req);
      }
    }

    return template;
  },

  // Add requirement item to template
  addRequirementToTemplate: async (templateId, requirementData) => {
    const itemQuery = `
      INSERT INTO document_requirement_items (
        template_id, requirement_name, requirement_description,
document_category,
        collection_stage, priority_level, is_mandatory, accepted_file_types,
        max_file_size, submission_instructions, help_text, display_order
      ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12)
      RETURNING *
    `;

    const itemValues = [
      templateId,
      requirementData.requirement_name,
      requirementData.requirement_description || '',
      requirementData.document_category || 'general',
      requirementData.collection_stage || 1,
      requirementData.priority_level || 'medium',
      requirementData.is_mandatory !== false,
      requirementData.accepted_file_types || ['pdf', 'docx', 'xlsx'],
      requirementData.max_file_size || 10485760,
      requirementData.submission_instructions || '',
      requirementData.help_text || '',
      requirementData.display_order || 0
    ];

    const result = await db.query(itemQuery, itemValues);
    return result.rows[0];
  },

  // Get available templates
  getRequirementTemplates: async (userCompanyId, industryCategory = null) => {
    let templatesQuery = `
      SELECT
        drt.*,
```

```
        COUNT(dri.id) as requirement_count,
        COUNT(CASE WHEN dri.collection_stage = 1 THEN 1 END) as stage_1_count,
        COUNT(CASE WHEN dri.collection_stage = 2 THEN 1 END) as stage_2_count
      FROM document_requirement_templates drt
      LEFT JOIN document_requirement_items dri ON drt.id = dri.template_id
      WHERE drt.user_company_id = $1 AND drt.is_active = true
    `;

    const queryParams = [userCompanyId];

    if (industryCategory) {
      templatesQuery += ' AND drt.industry_category = $2';
      queryParams.push(industryCategory);
    }

    templatesQuery += ' GROUP BY drt.id ORDER BY drt.is_default DESC,
drt.template_name ASC';

    const result = await db.query(templatesQuery, queryParams);
    return result.rows;
  }
};

// Utility functions
const calculateDueDate = (stage, priority) => {
  const now = new Date();
  let daysToAdd = 30; // Default 30 days

  // Adjust based on stage
  if (stage === 1) {
    daysToAdd = 21; // Stage 1: 3 weeks
  } else if (stage === 2) {
    daysToAdd = 14; // Stage 2: 2 weeks (more urgent)
  }

  // Adjust based on priority
  switch (priority) {
    case 'critical':
      daysToAdd = Math.floor(daysToAdd * 0.5); // Half the time
      break;
    case 'high':
      daysToAdd = Math.floor(daysToAdd * 0.75); // 75% of time
      break;
    case 'low':
      daysToAdd = Math.floor(daysToAdd * 1.5); // 150% of time
      break;
  }
```

```
  now.setDate(now.getDate() + daysToAdd);
  return now;
};

const calculateReminderDate = (dueDate) => {
  const reminderDate = new Date(dueDate);
  reminderDate.setDate(reminderDate.getDate() - 7); // Remind 1 week before due
date
  return reminderDate;
};

const checkPipelineProgression = async (targetCompanyId, canProgressToOffer,
canProgressToLegal) => {
  // Get current pipeline stage
  const company = await db.query('SELECT stage FROM target_companies WHERE id =
$1', [targetCompanyId]);
  if (company.rows.length === 0) return;

  const currentStage = company.rows[0].stage;

  // Check if we can progress to offer preparation
  if (canProgressToOffer && currentStage === 'Due Diligence') {
    // Automatically progress to offer preparation stage
    await db.query(
      'UPDATE target_companies SET stage = $1, updated_at = NOW() WHERE id =
$2',
      ['Offer Preparation', targetCompanyId]
    );

    // Log stage change
    await logActivity(targetCompanyId, null, 'stage_changed', 'Automatically
progressed to Offer Preparation - Stage 1 documents completed');
  }

  // Check if we can progress to legal & accounting
  if (canProgressToLegal && currentStage === 'Heads of Terms') {
    // Automatically progress to legal & accounting stage
    await db.query(
      'UPDATE target_companies SET stage = $1, updated_at = NOW() WHERE id =
$2',
      ['Legal & Accounting', targetCompanyId]
    );

    // Log stage change
    await logActivity(targetCompanyId, null, 'stage_changed', 'Automatically
progressed to Legal & Accounting - Stage 2 documents completed');
```

```
  }
};

const notifyWaiverApprovers = async (targetCompanyId, waiverId,
requirementName) => {
  // Get all users who can approve waivers (company admins and deal managers)
  const approversQuery = `
    SELECT DISTINCT u.id
    FROM users u
    JOIN user_deal_assignments uda ON u.id = uda.user_id
    WHERE uda.target_company_id = $1
    AND uda.is_active = true
    AND (u.is_company_admin = true OR u.user_role IN ('company_admin',
'deal_manager'))
  `;

  const approvers = await db.query(approversQuery, [targetCompanyId]);

  // Create notifications for all approvers
  for (const approver of approvers.rows) {
    await createTeamNotification(
      approver.id,
      'waiver_approval_required',
      'Document Waiver Approval Required',
      `Waiver requested for document requirement: ${requirementName}`,
      targetCompanyId,
      null
    );
  }
};
```

File Structure Updates

None

```
routes/
├── document-requirements.js (new)

public/
├── js/
│   └── staged-requirements.js (new)
└── css/
    └── staged-requirements.css (new)

views/
├── document-requirements/
```

```
|   ├── company-requirements.ejs (new)
|   ├── stage-progression.ejs (new)
|   └── template-manager.ejs (new)
```

Success Criteria for Prompt 12a

- ☐ Two-stage document collection system operational
- ☐ Industry-specific document requirement templates
- ☐ Pipeline integration prevents premature stage progression
- ☐ Document requirement customization and waiver system
- ☐ Stage progression tracking and analytics
- ☐ Template management for different industries and deal sizes
- ☐ Automatic due date calculation and reminder scheduling
- ☐ Comprehensive requirement status tracking
- ☐ Integration with existing portal and document systems
- ☐ Professional requirement management interface

Business Value

This prompt enables:

- **Structured Due Diligence** - Systematic two-stage document collection process
- **Industry Expertise** - Customized requirements based on industry best practices
- **Process Control** - Pipeline progression tied to document completion
- **Risk Management** - Waiver system with proper approval and justification
- **Efficiency Gains** - Automated tracking and reminder systems

Integration with Existing System

This prompt integrates seamlessly with:

- **Prompts 11a & 11b**: Uses portal system for document submission
- **Pipeline Management**: Controls stage progression based on document completion
- **Team Management**: Waiver approvals and notifications
- **Future Chase-up System**: Provides foundation for automated reminders

The staged document requirements system ensures that M&A deals progress through proper due diligence phases with appropriate documentation at each stage, maintaining professional standards while providing flexibility for exceptional circumstances!